UADE

UADE

Programación III – Diseño y Análisis de Algoritmos

Temas a desarrollar

1. Repaso Clase Anterior
2. Introducción a Backtracking
3. Problema de las 8 reinas
4. Problema suma de subconjuntos
5. Problema de partición en mitades

Repaso Clase Anterior

UADE

Algoritmo de Floyd

- El algoritmo de Floyd consiste en la búsqueda del costo de caminos mínimos entre cualquier par de nodos de un grafo dirigido con costos positivos, siempre que este existan esos posibles caminos. En el caso que no exista camino entre dos nodos, el resultado arrojará la no existencia del mismo, pero no afectará el resultado del resto de los caminos.

- El algoritmo sigue una estrategia por Programación Dinámica que se puede representar con la siguiente función:

$$M_k[i,j] = \text{mín}(M_{k-1}[i,j], M_{k-1}[i,k] + M_{k-1}[k,j])$$

- La complejidad temporal de algoritmo es de Θ (n^3), siendo n la cantidad de nodos del grafo.

Introducción a Backtracking

Backtracking

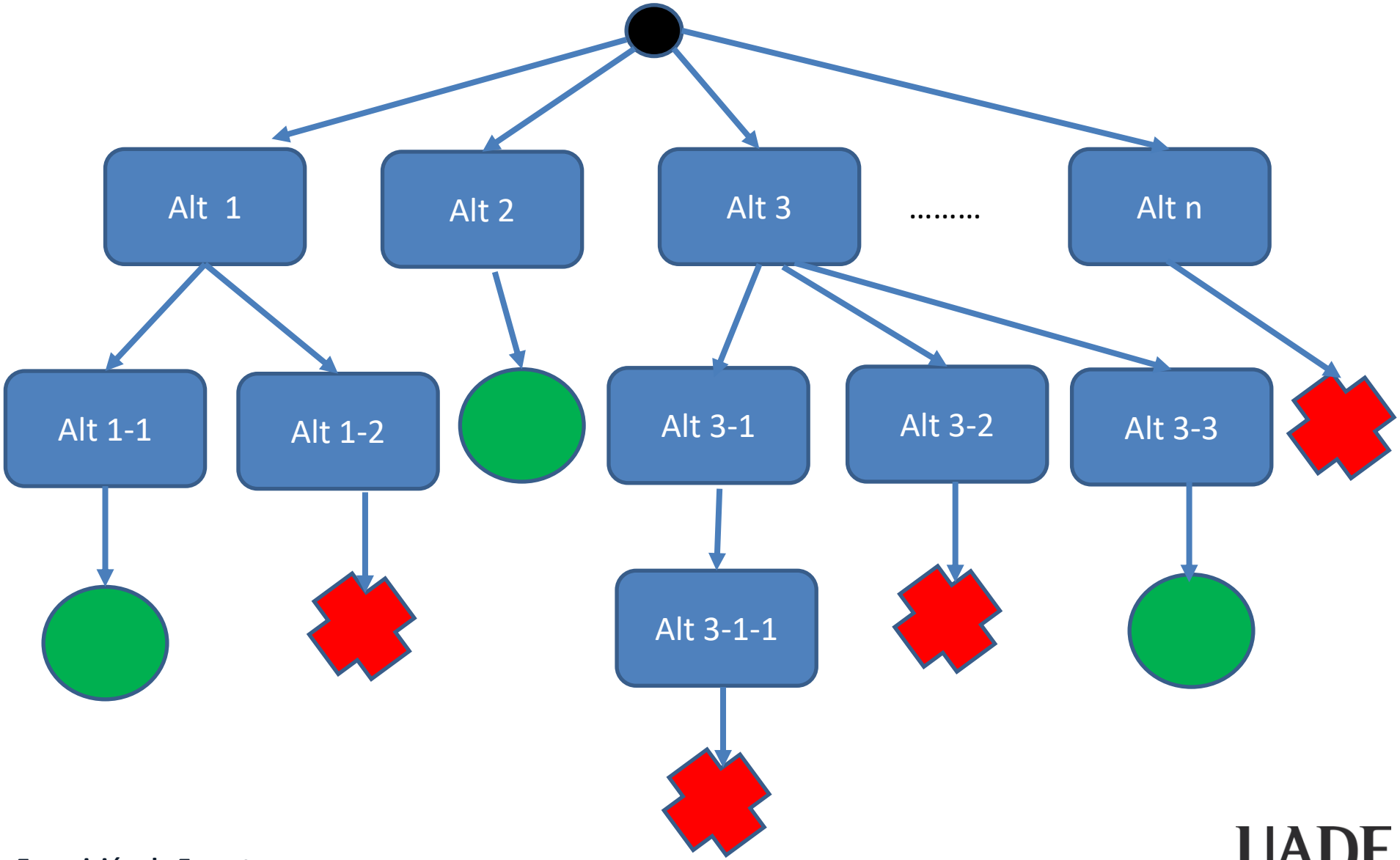
La solución posible se basa es la exploración directa de todas las posibilidades

Es un método de búsqueda de soluciones exhaustivas sobre grafos dirigidos acíclicos

Podado de alternativa

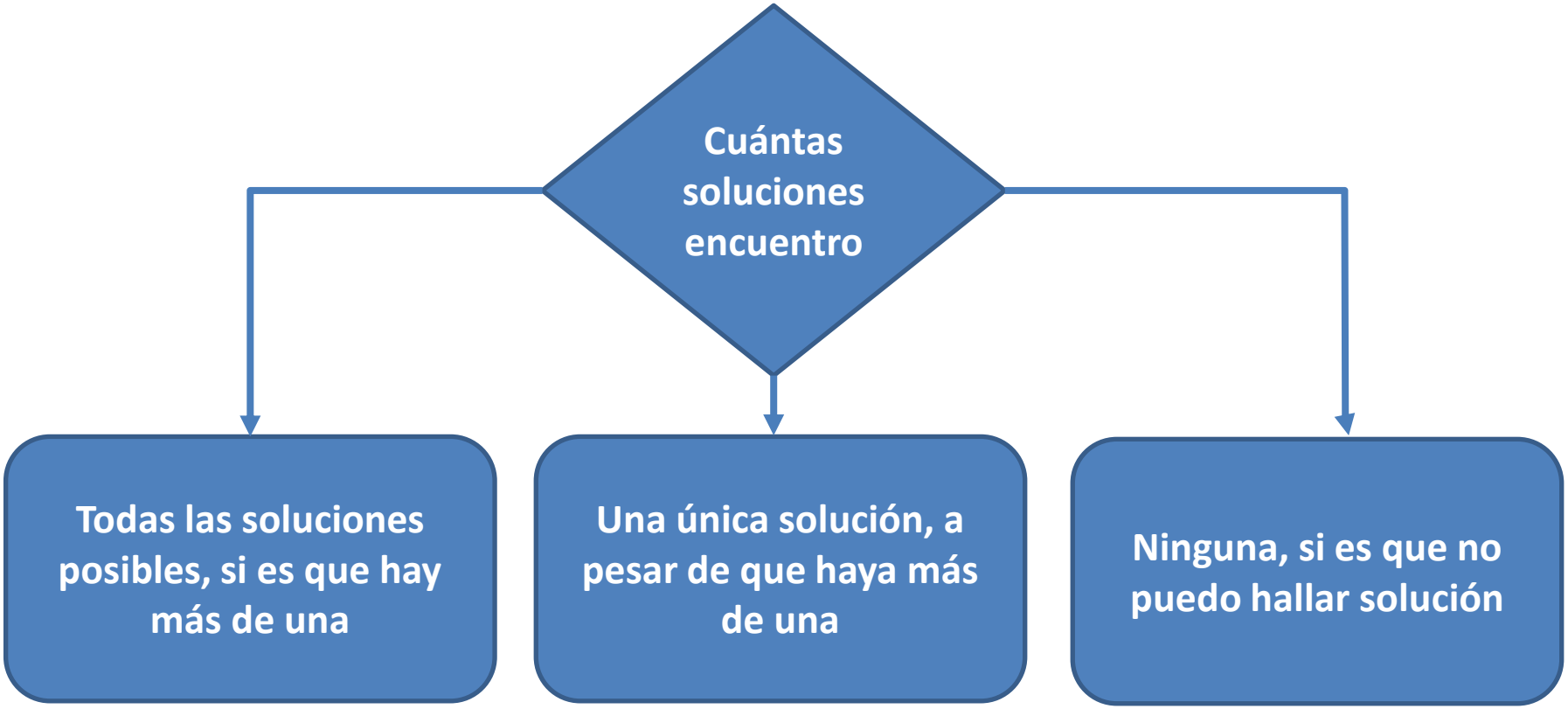
Cuando hablamos de grafos nos referimos a una construcción lógica de posibles alternativas a evaluar

Backtracking



UADE

Backtracking - Soluciones



Backtracking – Esquema General

ALGORITMO BACKTRACKING

Entrada: S: Solución parcial, E: etapa

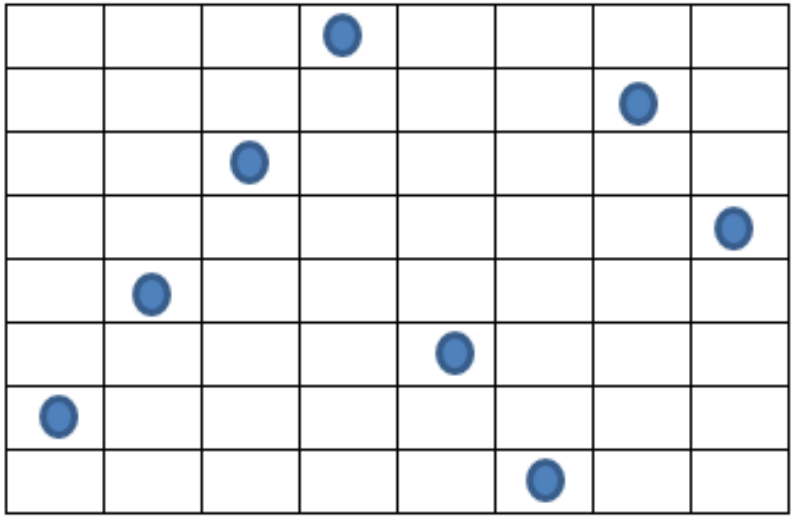
```
Valores ← generarValoresPosibles(S, E)
para cada v ∈ Valores
   agregar(S, E, v)
   si factible(S)
      si esSolucion(S)
         mostrar(S)
      sino
         backtracking(S, E + 1)
      fin si
   fin si
   eliminar(S, E)
fin para
```

Problema de las 8 reinas

Problema de las 8 reinas

"Dado un tablero de NxN ubicar N reinas de modo tal que no se coman entre sí. Una reina come a otra si esta en la misma fila, columna o diagonal. Encontrar una solución para el problema."

Para nuestro caso el n considerado será 8, y una posible solución es:



Problema de las 8 reinas - Estrategia

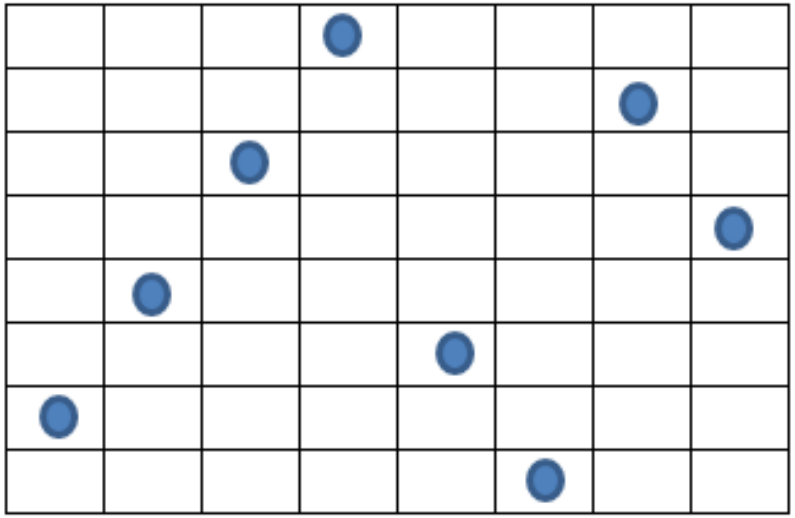
- La estrategia de solución por *backtracking* es ir probando de colocar una reina en la primera fila de la columna 1, luego ubicar la segunda reina en la fila 2, en la primera columna factible, y así sucesivamente.

- Si no se llega a usa solución se vuelve atrás y se prueba colocando una reina en una fila pero en columna distinta, por ejemplo, si para la columna 1 de la fila 1 no se llega a una solución, se prueba colocándola en la columna 2, y así hasta llegar a una solución.

Problema de las 8 reinas

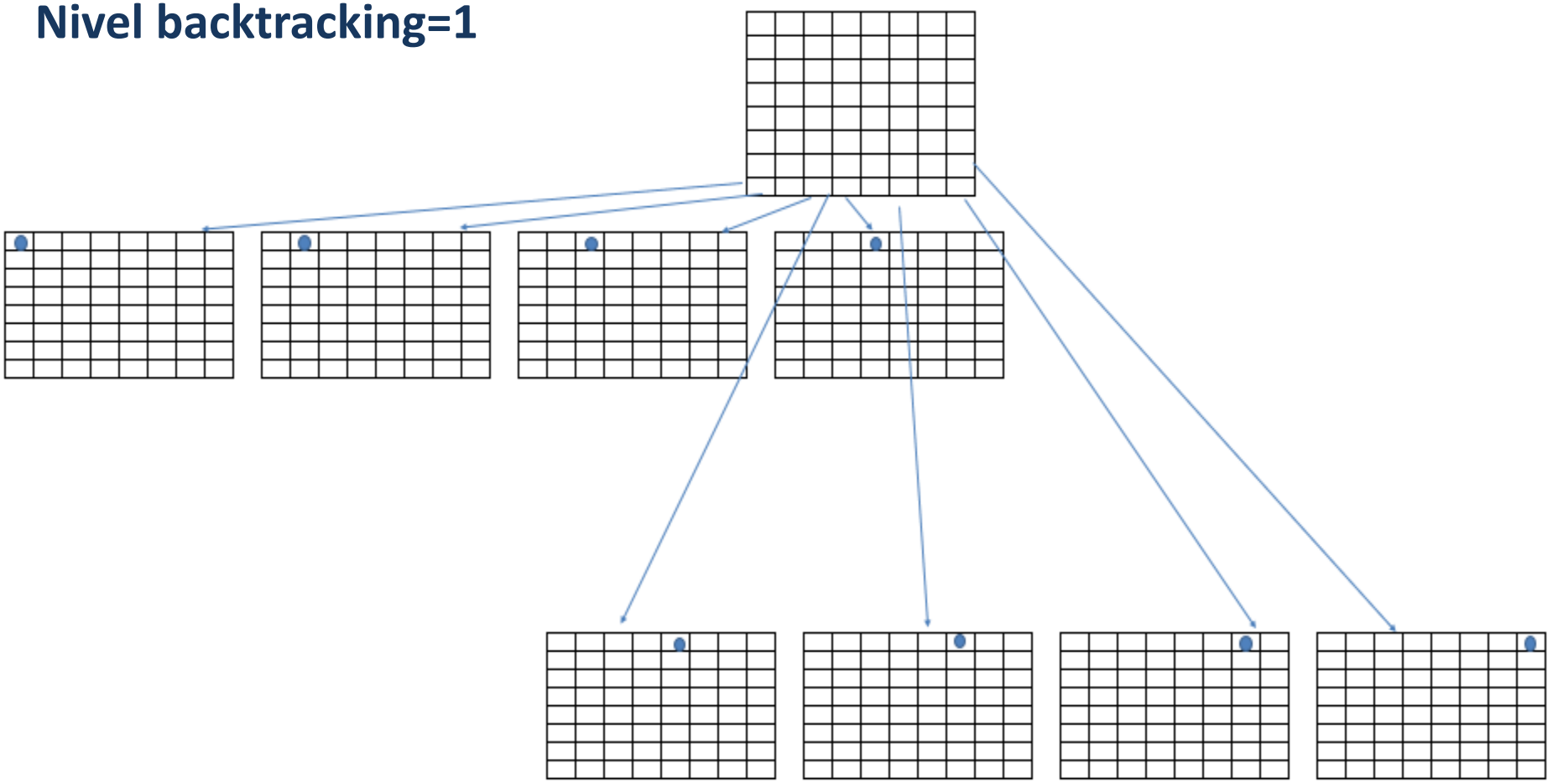
"Dado un tablero de NxN ubicar N reinas de modo tal que no se coman entre sí. Una reina come a otra si esta en la misma fila, columna o diagonal. Encontrar una solución para el problema."

Para nuestro caso el n considerado será 8, y una posible solución es:



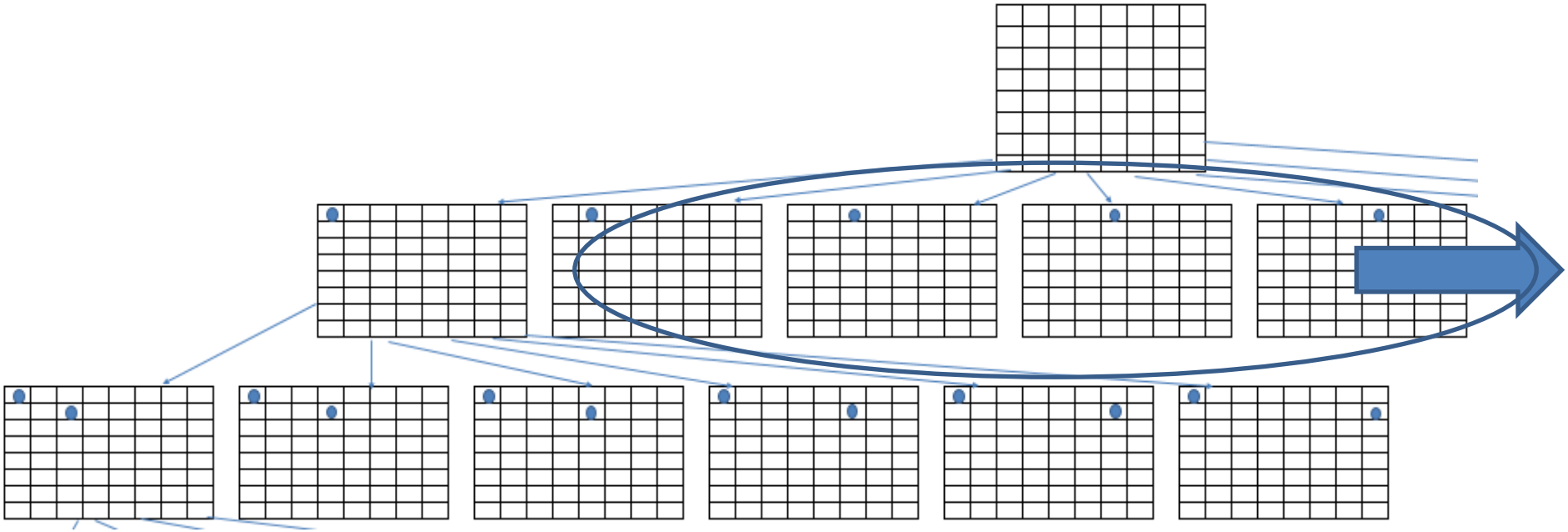
Problema de las 8 reinas

Nivel backtracking=1



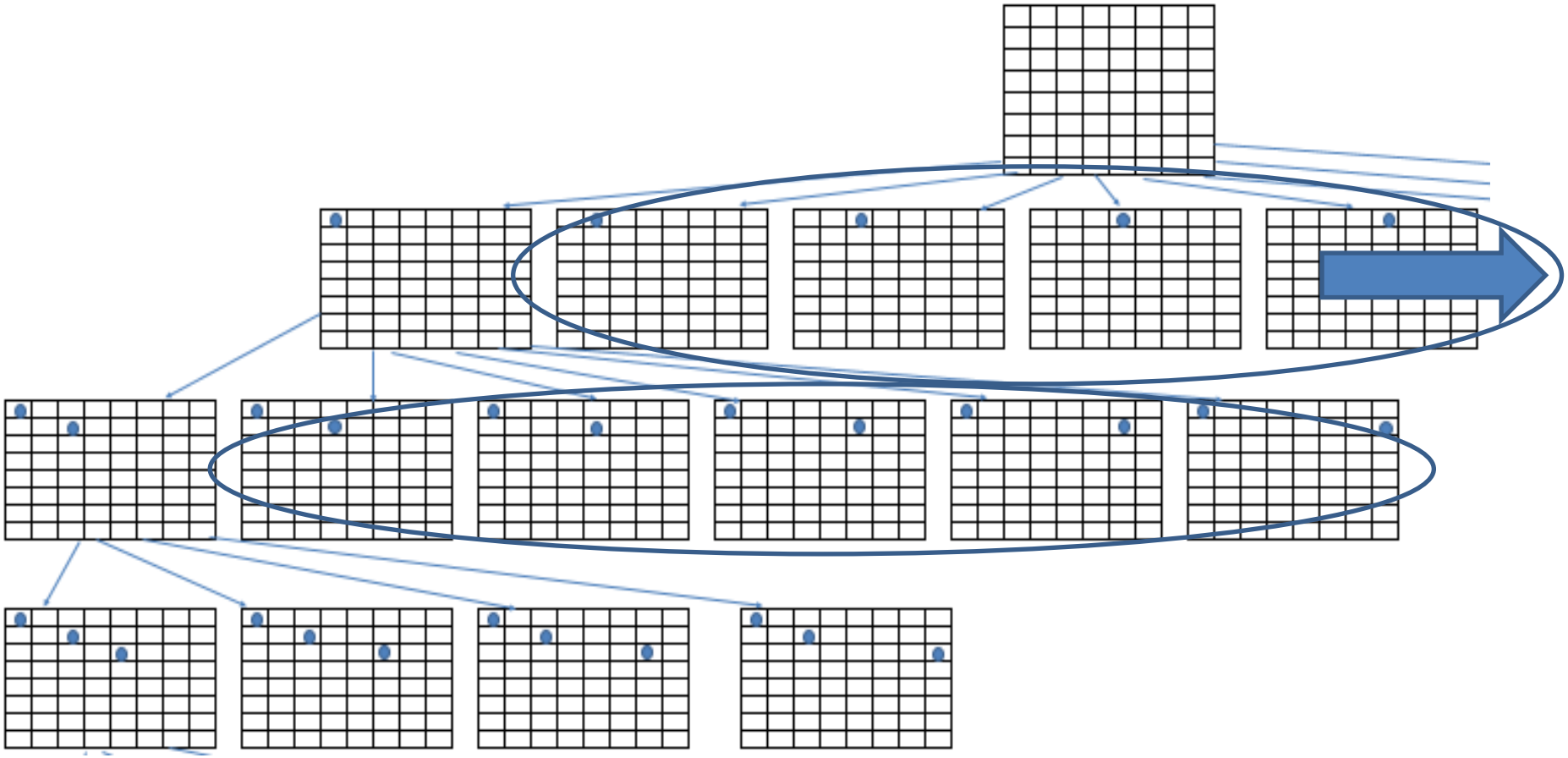
Problema de las 8 reinas

Nivel backtracking =2



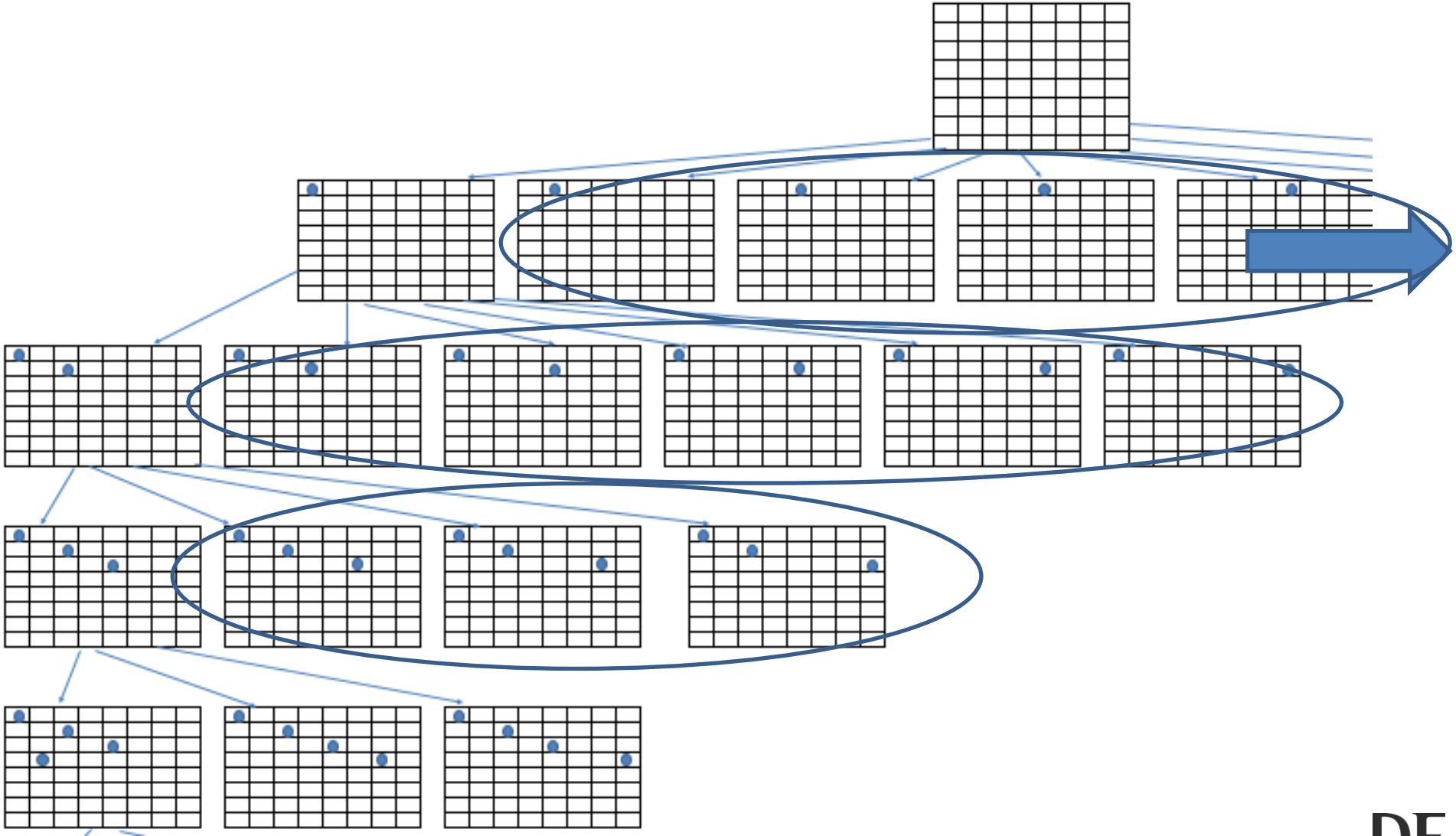
Problema de las 8 reinas

Nivel backtracking =3



Problema de las 8 reinas

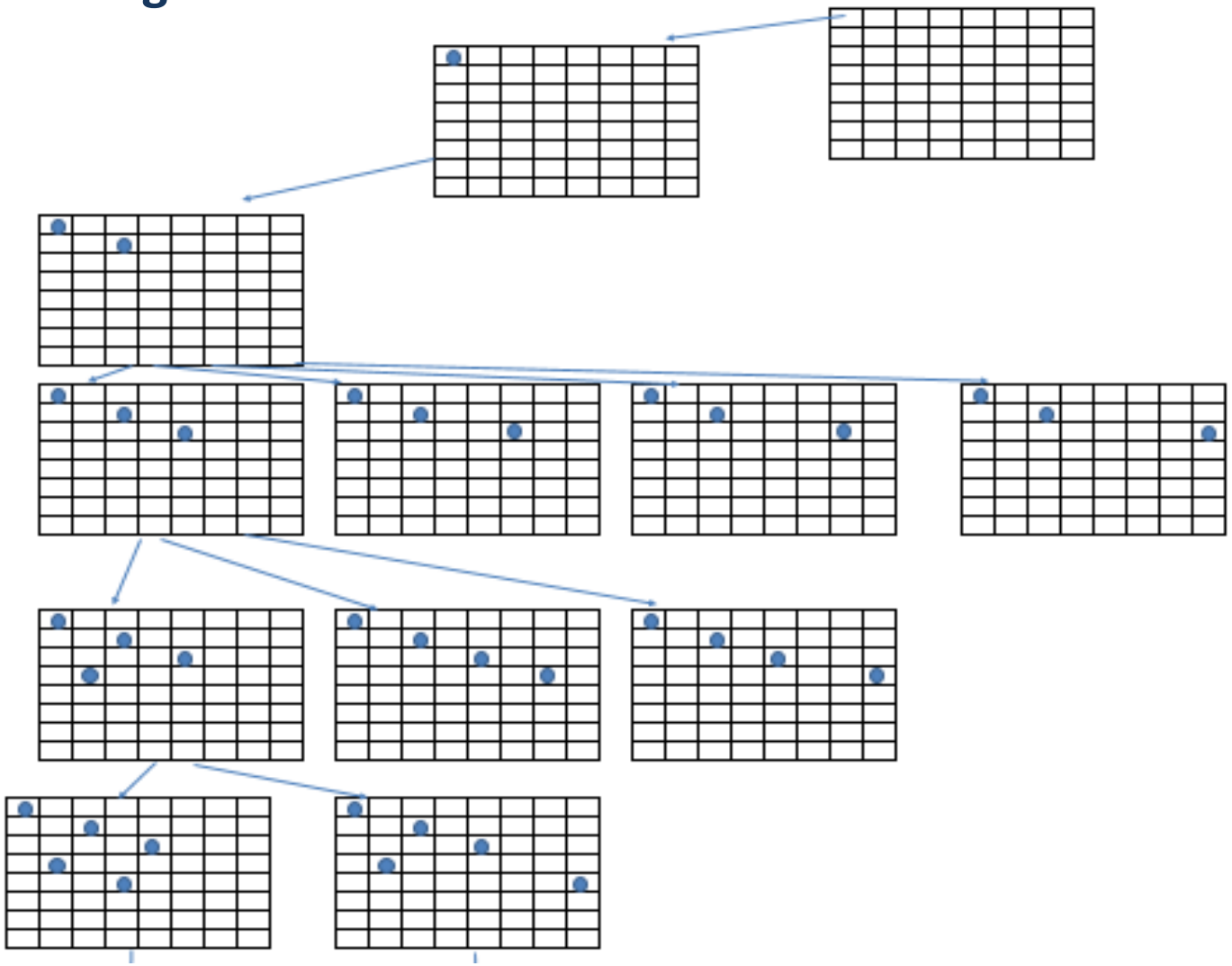
Nivel backtracking =4



UADE

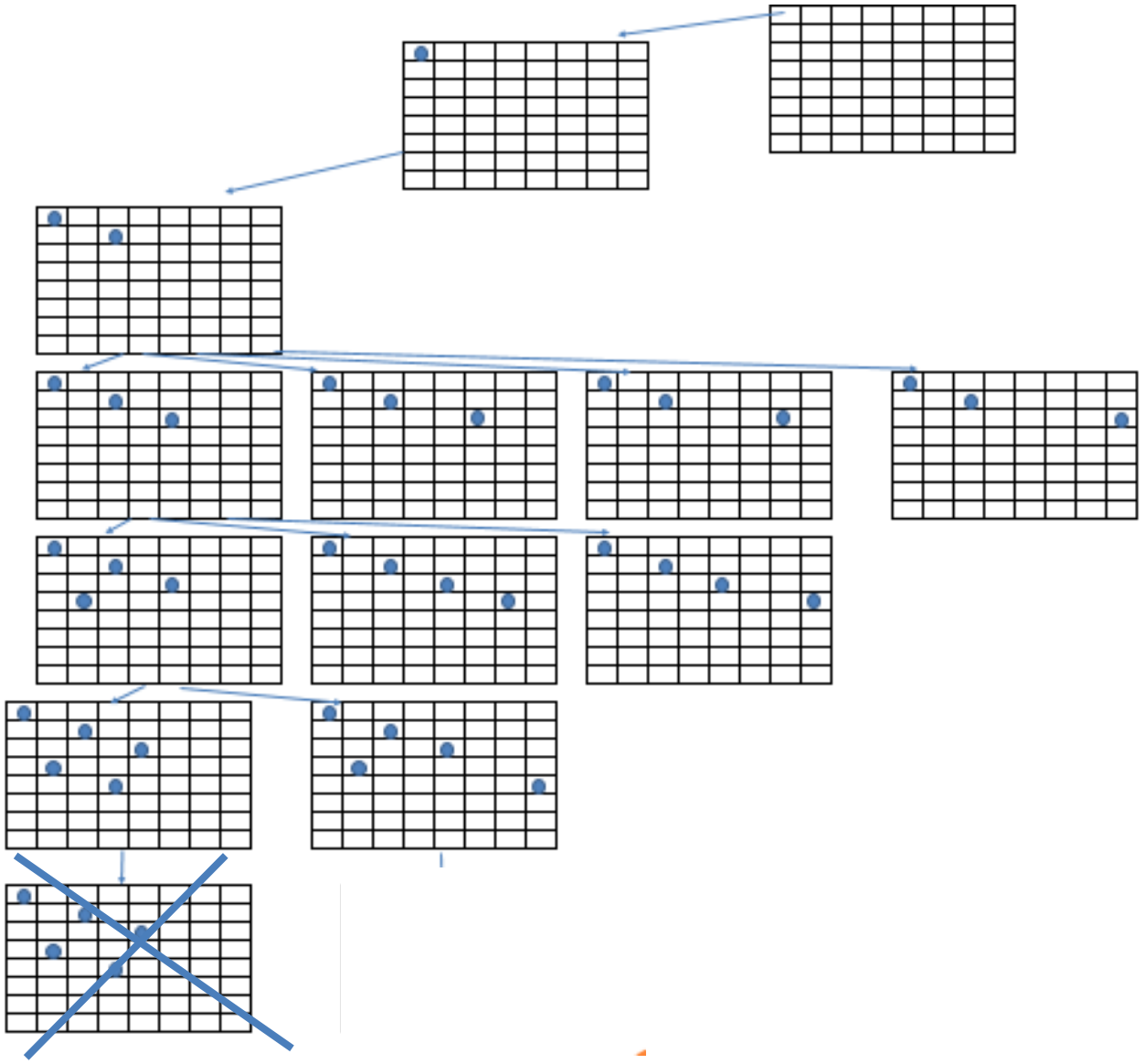
Problema de las 8 reinas

Nivel backtracking =5



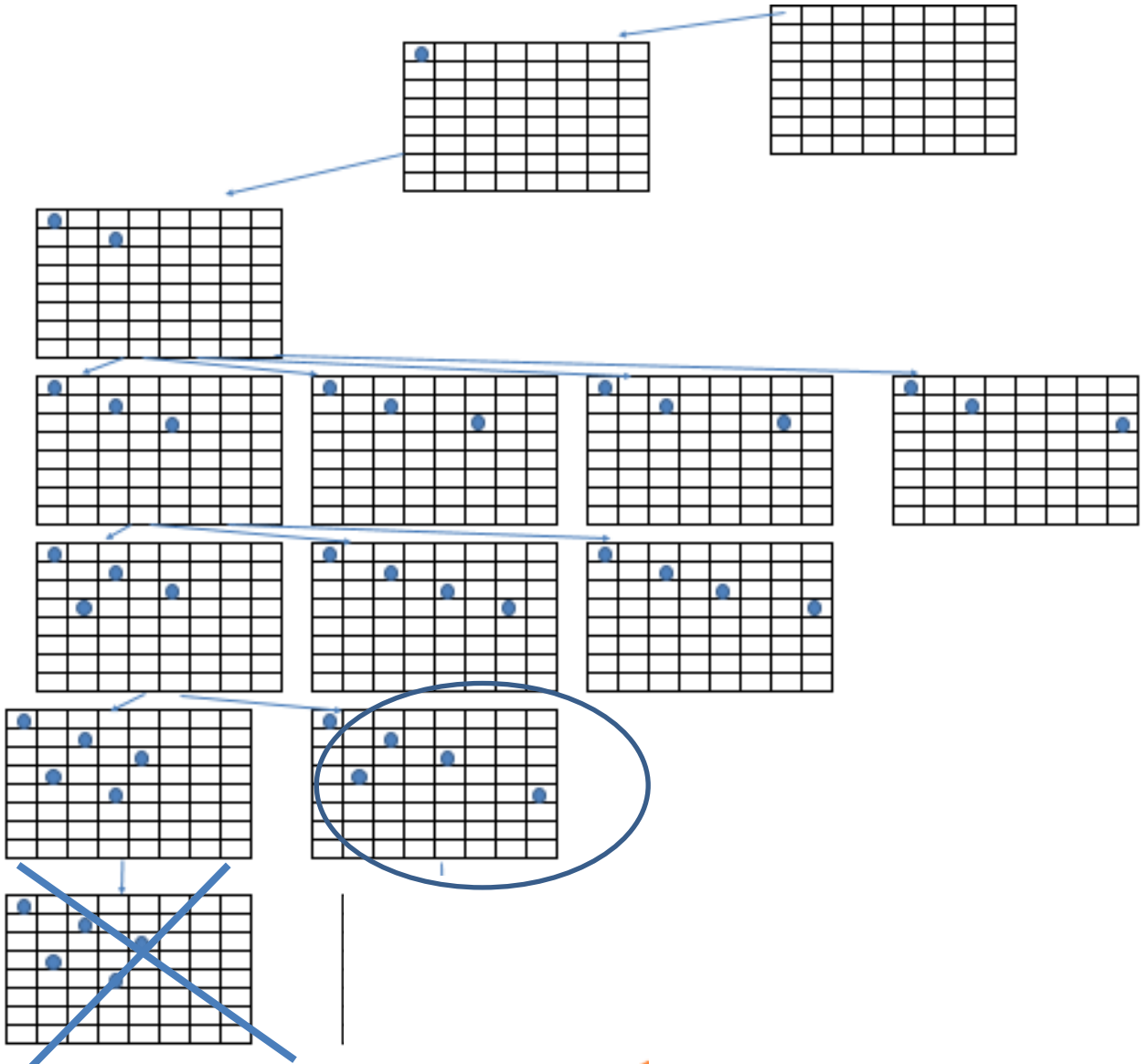
Problema de las 8 reinas

Nivel backtracking =6



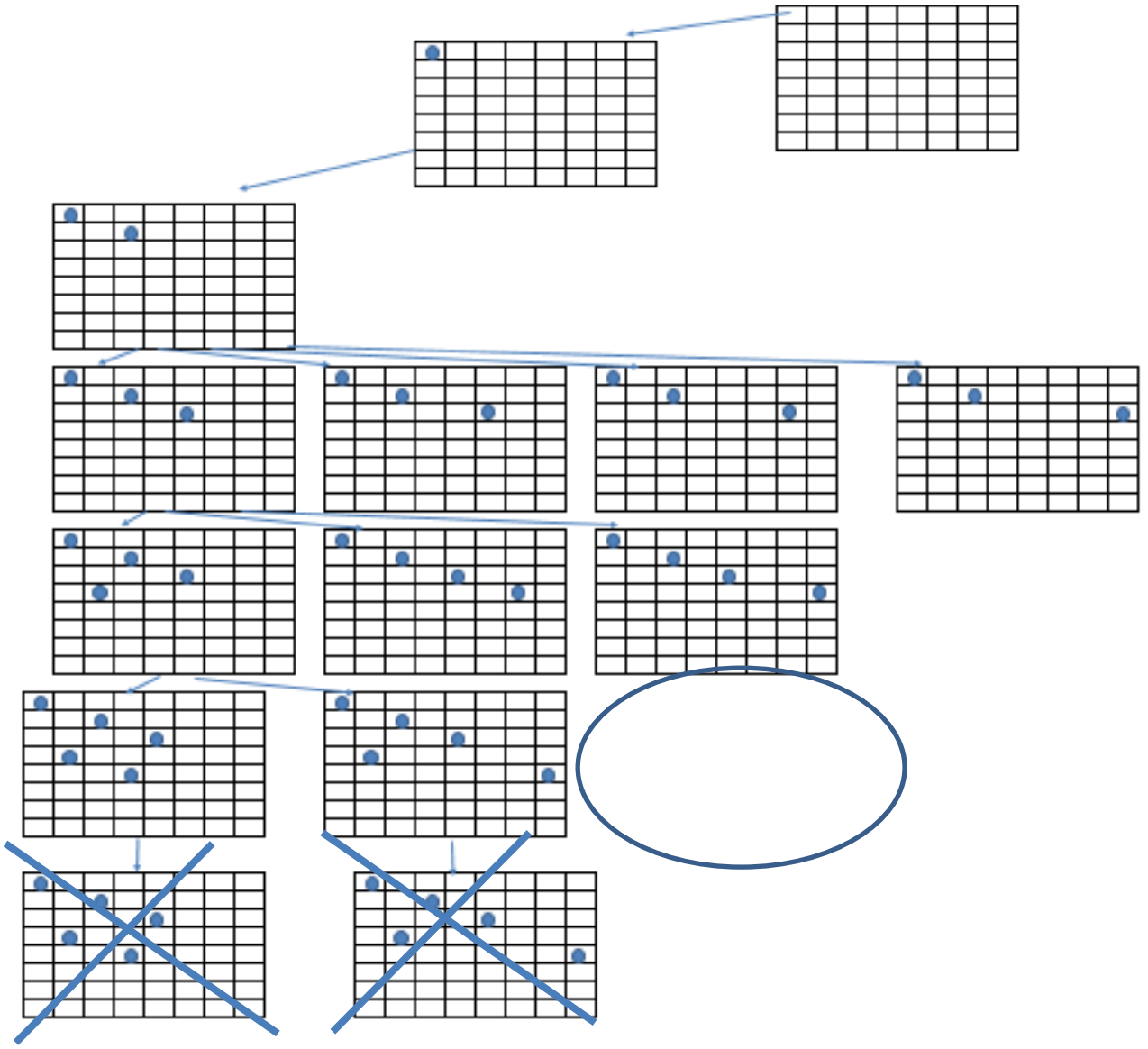
Problema de las 8 reinas

Nivel backtracking =5



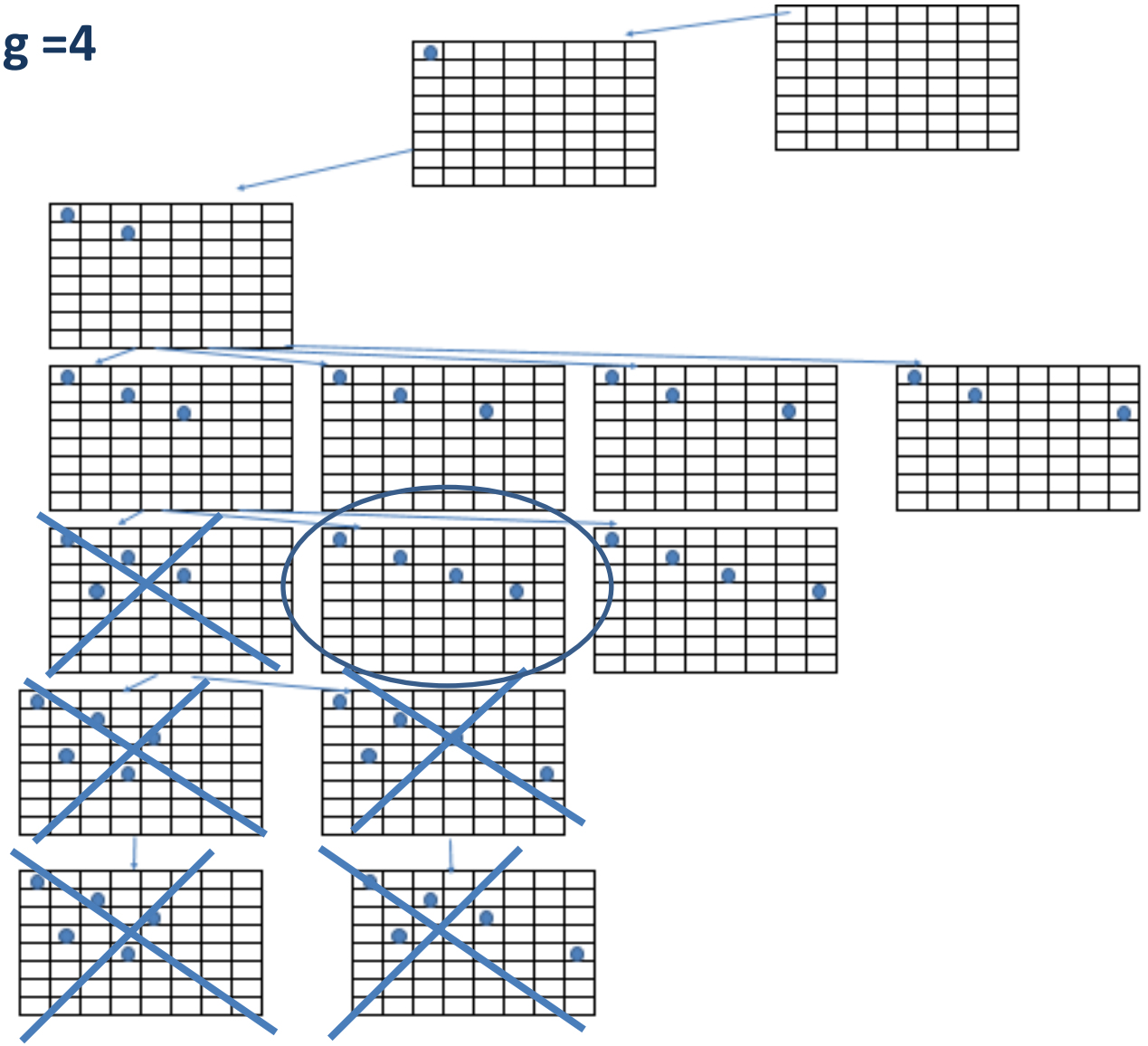
Problema de las 8 reinas

Nivel backtracking =5



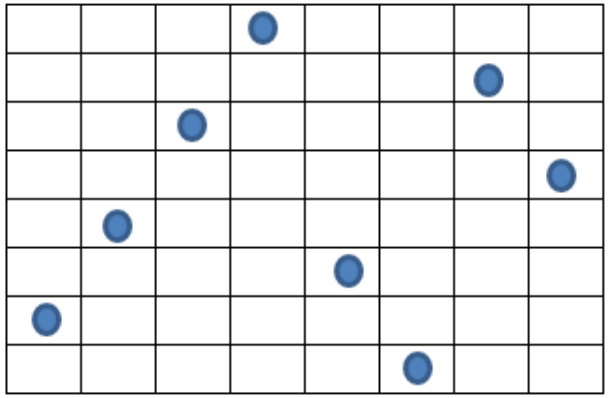
Problema de las 8 reinas

Nivel backtracking =4



Problema de las 8 reinas - Solución

- La solución trivial sería una matriz booleana con valores verdaderos en las posiciones donde están las reinas. Como esta matriz sería rala, es decir, tendría N valores en uso de los NxN disponibles.



- Teniendo en cuenta que nunca habrá mas de una reina en cada fila, se puede considerar un vector de N posiciones que indica para cada fila, en que columna se ubicara la reina. Es decir que el vector deberá contener los números del 1 al N.

4	7	3	8	2	5	1	6
1	2	3	4	5	6	7	8

Problema de las 8 reinas - Solución

Chequeo Filas	No se puede tener mas de una reina en la misma fila por definición de la estructura.
Chequeo Columnas	Controlar que no aparezcan números repetidos en cada posición del vector
Chequeo Diagonal	Dada una fila i y una columna j donde estamos probando colocar una reina, las diagonales de esta ubicación estarán k filas mas arriba (1 <= k < i) y desplazadas k columnas a derecha o a izquierda.

Problema de las 8 reinas - Algoritmo

ALGORITMO REINAS

Entrada: S: Vector<entero>, E: entero, n:entero
Salida: verdadero-falso

```
booleano solucion ←falso
S[E] ← 0
mientras S[E] < n − 1 Y NO solucion
    S[E] ← S[E] + 1
    si ReinaValida(S, E)
        si E = n − 1
            solucion ←verdadero
        sino
            solucion ← Reinas(S, E + 1, n)
        fin si
    fin si
fin mientras
devolver solucion
```

ALGORITMO REINAVALIDA

Entrada: S: Vector<entero>, E: entero
Salida: verdadero-falso

```
para i = 0 hasta E − 1
    si S[i] = S[E] O |S[i] − S[E]| = |i − E|
        devolver falso
    fin si
fin para
devolver verdadero
```

Problema de las 8 reinas – Complejidad Temporal

ALGORITMO REINAS

Entrada: S: Vector<entero>, E: entero, n:entero
Salida: verdadero-falso
  booleano *solucion* ←falso
  $S[E] \leftarrow 0$
  mientras $S[E] < n-1$ **Y NO** *solucion*
    $S[E] \leftarrow S[E]+1$
    si $ReinaValida(S, E)$
      si $E = n-1$
        solucion ←verdadero
      sino
        $solucion \leftarrow Reinas(S, E+1, n)$
      fin si
    fin si
  fin mientras
  devolver *solucion*

a=n; b=1; k=1

$$T(n) \in \begin{cases} \Theta(n^k) & \text{si } a < 1 \\ \Theta(n^{k+1}) & \text{si } a = 1 \\ \Theta(a^{n \text{ div } b}) & \text{si } a > 1 \end{cases}$$

$$\mathcal{O}(n^n)$$

ALGORITMO REINAVALIDA

Entrada: S: Vector<entero>, E: entero
Salida: verdadero-falso
  para $i = 0$ **hasta** $E-1$
    si $S[i] = S[E]$ **O** $|S[i] - S[E]| = |i - E|$
      devolver falso
    fin si
  fin para
  devolver verdadero

$$\mathcal{O}(n)$$

Problema Suma de Subconjuntos

Suma de Subconjuntos

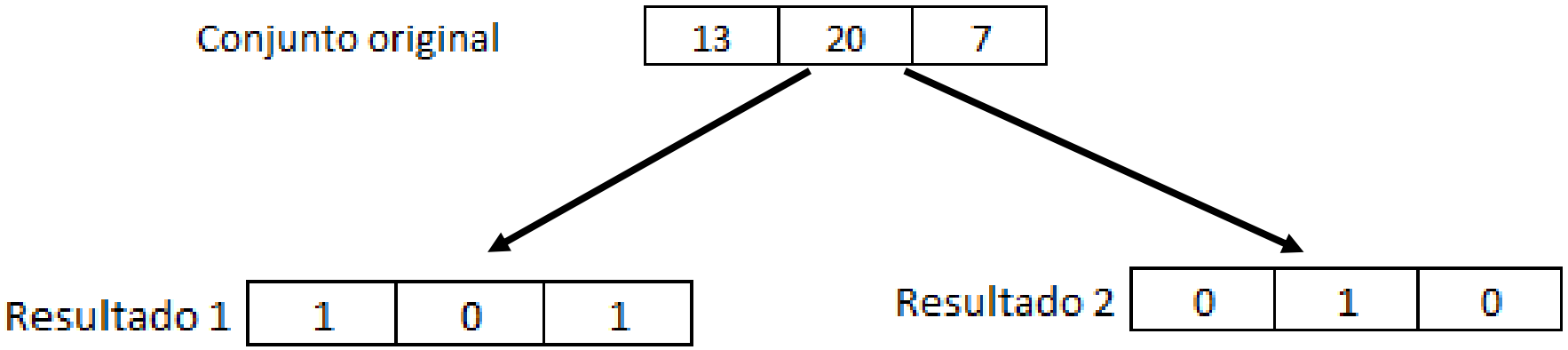
"Dado un conjunto de números C y un valor v, mostrar todos los subconjuntos de C en los cuales la suma de sus elementos sea v."

Por ejemplo, si C= {5,3,2,6,1,4} y v = 6, los posibles subconjuntos son:

{5,1}
{3,2,1}
{6}
{2,4}

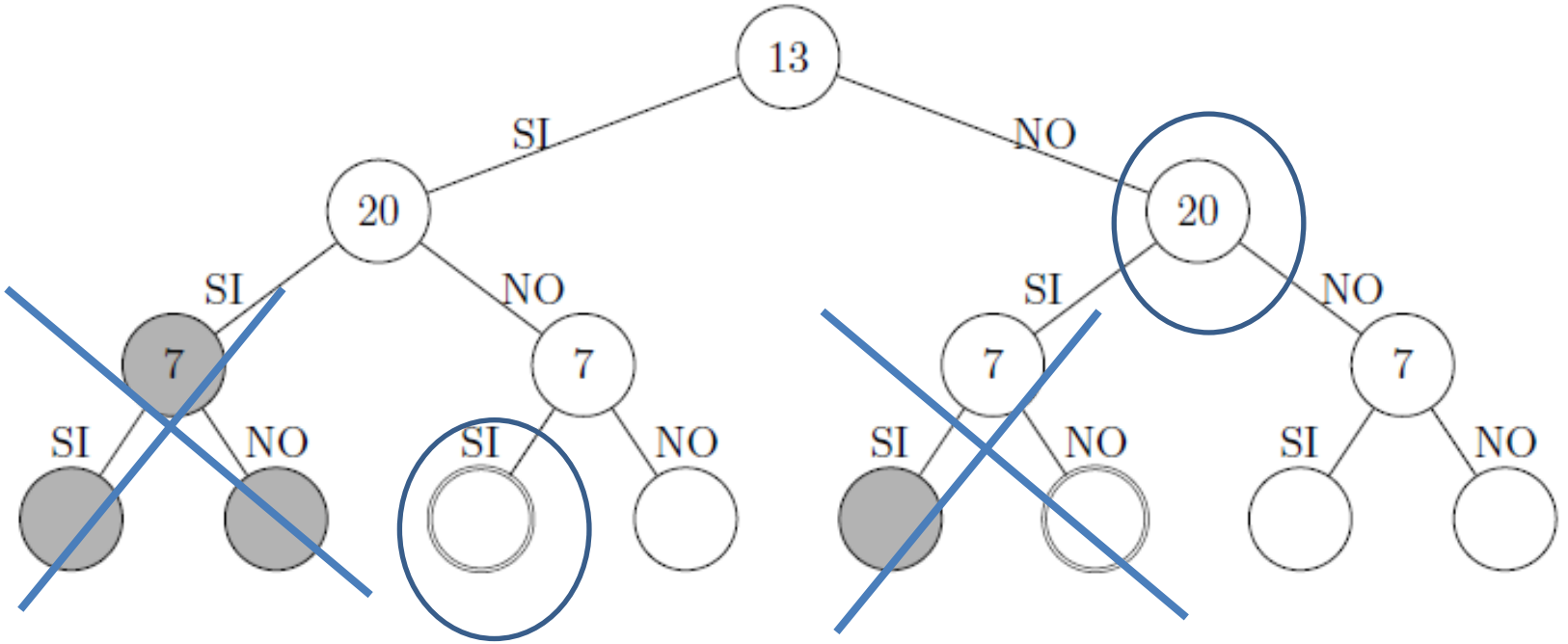
Suma de Subconjuntos - Estrategia

En una posible solución al problema, cada elemento de C puede estar o no estar, con lo cual podremos tener una vector de 0s y 1s o un vector booleano con dimensión igual a la cardinalidad de C.
Supongamos **C = {13; 20; 7}**



Suma de Subconjuntos - Estrategia

Si armamos el árbol completo del ejemplo para **C = {13; 20; 7} y v = 20** el árbol será el siguiente:



UADE

Suma de Subconjuntos - Algoritmo

```
ALGORITMO SUMAM
Entrada: v: Vector<entero>, m: entero, ActSolucion: Vector<entero>,
  ActSuma: entero, etapa: entero
  para i = 0 hasta 1
    ActSolucion[etapa] ← i
    ActSuma ← ActSuma + v[etapa] * i
    si etapa = tamanio(v)
      si ActSuma = m
        mostrar(ActSolucion)
      fin si
    sino
      si ActSuma < m
        sumaM(v, m, ActSolucion, ActSuma, etapa + 1)
      fin si
    fin si
  fin para
```

Suma de Subconjuntos – Complejidad Temporal

```
ALGORITMO SUMAM
Entrada: v: Vector<entero>, m: entero, ActSolucion: Vector<entero>,
  ActSuma: entero, etapa: entero
  para i = 0 hasta 1
    ActSolucion[etapa] ← i
    ActSuma ← ActSuma + v[etapa] * i
    si etapa = tamanio(v)
      si ActSuma = m
        mostrar(ActSolucion)
      fin si
    sino
      si ActSuma < m
        sumaM(v, m, ActSolucion, ActSuma, etapa + 1)
      fin si
    fin si
  fin para
```

a=2; b=1; k=0

$$T(n) \in \begin{cases} \Theta(n^k) & \text{si } a < 1 \\ \Theta(n^{k+1}) & \text{si } a = 1 \\ \Theta(a^{n \text{ div } b}) & \text{si } a > 1 \end{cases}$$

$$O(2^n)$$

Problema Partición en Mitades

Problema Partición en Mitades

Dado un conjunto de n enteros se desea encontrar, si existe, una partición en dos subconjuntos disjuntos, tal que la suma de sus elementos sea la misma. Si existe más de una, mostrar todas las posibles.

Ejemplo: ***C={2,4,5,6,1}***

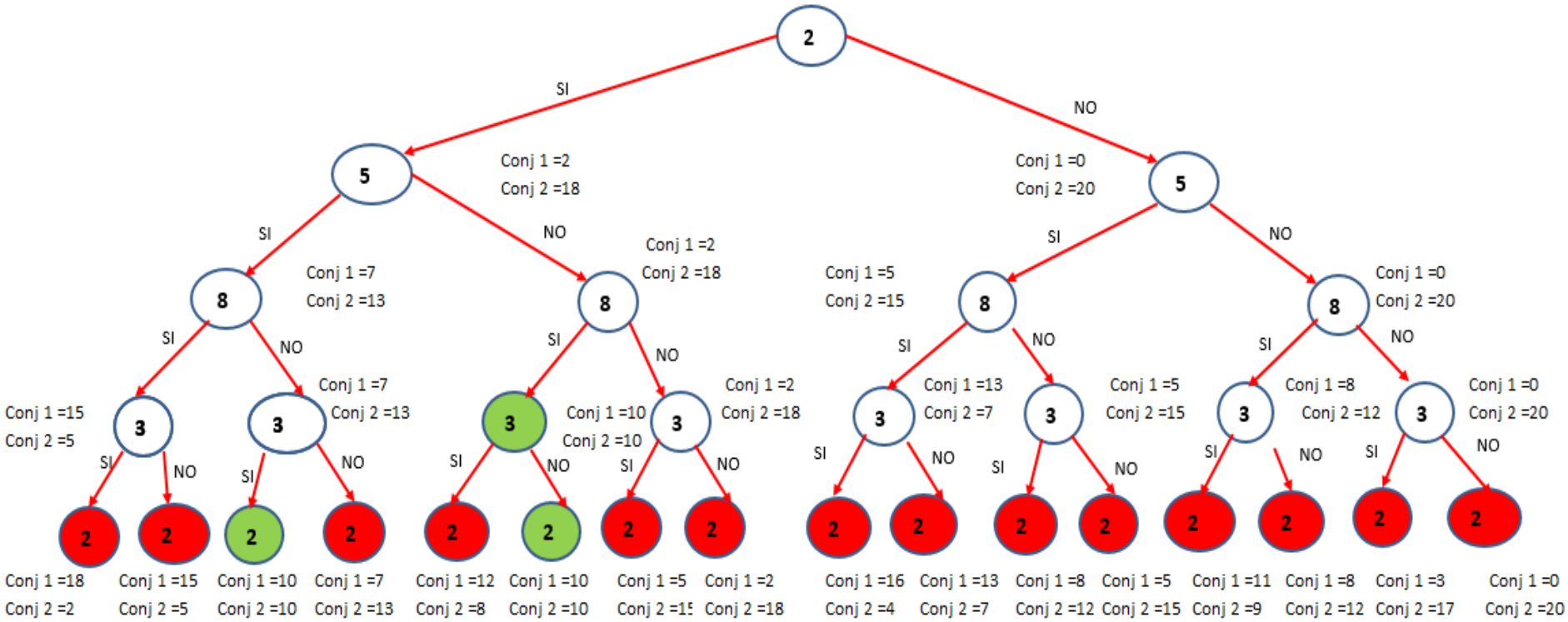
Resultado Partición-1 ***{2,6,1}=9*** *Partición-2* ***{4,5}=9***

Problema Partición en Mitades - Estrategia

- La estrategia por backtracking consistirá en colocar cada elemento en el conjunto 1 o en el conjunto 2, probando todas las alternativas.
- Cuando se tenga cada elemento en alguno de los conjuntos verificar si la suma de los elementos de cada uno de ellos es igual.
- En una estrategia un poco mas elaborada se podrá llevar el acumulado de cada conjunto y no hace falta hacer la suma al final.

Problema Partición en Mitades - Estrategia

- Supongamos el conjunto: {2,5,8,3, 2}



UADE

Problema Partición en Mitades - Algoritmo

```
ALGORITMO PARTICIONMITAD
Entrada: S: Vector<entero>, E: entero, v: Vector<entero>
  para i = 1 hasta 2
    S[E] ← i
    si E = longitud(v) − 1
      suma1 ← 0; suma2 ← 0
      para j = 0 hasta longitud(v) − 1
        si S[j] = 1
          suma1 ← suma1 + v[j]
        sino
          suma2 ← suma2 + v[j]
        fin si
      fin para
      si suma1 = suma2
        Mostrar(S)
      fin si
    sino
      particionMitad( S , E+1, v )
    fin si
  fin para
```

Partición en Mitades – Complejidad Temporal

```
ALGORITMO PARTICIONMITAD
Entrada: S: Vector<entero>, E: entero, v: Vector<entero>
  para i = 1 hasta 2
    S[E] ← i
    si E = longitud(v) − 1
      suma1 ← 0; suma2 ← 0
      para j = 0 hasta longitud(v) − 1
        si S[j] = 1
          suma1 ← suma1 + v[j]
        sino
          suma2 ← suma2 + v[j]
        fin si
      fin para
      si suma1 = suma2
        Mostrar(S)
      fin si
    sino
      particionMitad(v, S, E + 1)
    fin si
  fin para
```

a=2; b=1; k=1

$$T(n) \in \begin{cases} \Theta(n^k) & \text{si } a < 1 \\ \Theta(n^{k+1}) & \text{si } a = 1 \\ \Theta(a^{n \ div \ b}) & \text{si } a > 1 \end{cases}$$

$$\mathcal{O}(2^n)$$

Resolución problema práctica

UADE

Ejercicio

- Plantear la estrategia por backtracking
- Desarrollar pseudocódigo
- Calcular complejidad temporal

Ejercicio 1. Se tienen ***m*** procesadores idénticos y ***n*** tareas con un tiempo de ejecución dado. Se requiere encontrar una asignación de tareas a procesadores de manera de minimizar el tiempo de ejecución del total de tareas.

¡Muchas gracias!

UADE